API SECURITY RISK ANALYSIS REPORT

API Security

Risk Analysis

APIS TESTED	JSONPlaceholder + ReqRes
PREPARED BY	Shawn Ryan
DATE	March 23, 2026
TYPE	Read-Only Security Assessment
METHODOLOGY	OWASP API Security Top 10
TOOLS	Postman, Browser DevTools, curl

2 — **APIs Analyzed**
JSONPlaceholder & ReqRes

8 — **Risk Findings**
Across OWASP API Top 10

3 — **Critical / High**
Requiring immediate attention

100% — **Passive Analysis**
No exploitation performed

Scope: Public demo APIs only — JSONPlaceholder (jsonplaceholder.typicode.com) and ReqRes (reqres.in). Read-only analysis. No exploitation.

Why API Security Matters

APIs are the invisible backbone of modern software. Every mobile app, SaaS platform, and cloud service relies on APIs to function. An insecure API gives attackers direct access to your data, users, and business logic — often bypassing all other security controls.

Mobile Apps

Every feature in a mobile app — login, payments, data display — calls an API. Insecure APIs expose all users.

SaaS Platforms

SaaS tools expose APIs for integrations and automation. A single broken endpoint can leak all customer data.

Third-Party Integrations

Business systems share data via APIs. A weak link in any integration partner creates risk across all connected systems.

Automation & Bots

Attackers use scripts to probe APIs at scale — testing thousands of credential combinations or scraping all exposed data.

83% of web traffic is now API traffic (Akamai 2023)

$4.9M average data breach cost — many start with API flaws

API #1 OWASP API Top 10 — dedicated threat list since 2019

APIs Tested — Scope & Methodology

API 1 — JSONPlaceholder

Base URL:	`https://jsonplaceholder.typicode.com`
Purpose:	Free fake REST API for testing & prototyping
Auth:	None required — fully open
Format:	JSON
Endpoints:	`/users`, `/posts`, `/comments`, `/albums`, `/todos`, `/photos`
HTTP Methods:	GET, POST, PUT, PATCH, DELETE (all simulated)
Rate Limiting:	None observed
HTTPS:	Yes — TLS enforced

API 2 — ReqRes

Base URL:	`https://reqres.in/api`
Purpose:	Hosted REST API for front-end testing
Auth:	Token-based (optional on some endpoints)
Format:	JSON
Endpoints:	`/users`, `/login`, `/register`, `/unknown`
HTTP Methods:	GET, POST, PUT, PATCH, DELETE
Rate Limiting:	None observed
HTTPS:	Yes — TLS enforced

All 8 Findings at a Glance

01 CRITICAL API1:2023 [JSONPlaceholder]

Broken Object-Level Authorization — User PII exposed without auth

02 CRITICAL API2:2023 [JSONPlaceholder]

Missing Authentication — All endpoints fully open to public

03 HIGH API3:2023 [JSONPlaceholder]

Excessive Data Exposure — Full PII fields returned on all responses

04 HIGH API2:2023 [ReqRes]

Predictable Numeric User IDs Enable IDOR Attacks

05 HIGH API6:2023 [JSONPlaceholder]

No Rate Limiting — Endpoints susceptible to enumeration & abuse

06 MEDIUM API5:2023 [ReqRes]

No Input Validation on POST /register — Accepts arbitrary email formats

07 MEDIUM API8:2023 [Both]

Missing Security Headers — No CSP, X-Frame-Options, or HSTS

08 LOW API9:2023 [JSONPlaceholder]

Verbose Error Messages Expose Internal Stack Information

Section 1: Critical & High Findings

Findings 01 – 05: Broken authorization, missing auth, excessive exposure, IDOR, no rate limiting

Findings 01 & 02 — Broken Authorization + Missing Authentication

01 CRITICAL Broken Object-Level Authorization

`Endpoint: GET /users/{id} — no auth required`

RISK

Any unauthenticated caller can retrieve any user's full record by changing the ID. User 1, 2, 3... all exposed. No token, no session, no check.

BUSINESS IMPACT

Full name, email, phone, website, company, address, and geo-coordinates exposed for all users. In production this enables mass harvesting of customer PII. Direct regulatory violation (GDPR/CCPA).

REMEDIATION

Require auth token on ALL /users endpoints. Enforce object-level checks: authenticated user can only access their own record unless admin role is present.

02 CRITICAL Missing Authentication on All Endpoints

`Endpoint: ALL endpoints — zero auth required`

RISK

Every single endpoint (users, posts, comments, todos, albums) accepts requests with no Authorization header. No API key, no bearer token, no session cookie is required or checked.

BUSINESS IMPACT

A banking, health, or SaaS system built on this architecture would have zero access control. All data — including write operations (POST/PUT/DELETE) — is accessible to anyone worldwide with an HTTP client.

REMEDIATION

Implement JWT or OAuth 2.0 bearer token authentication on all endpoints. Reject 401 Unauthorized for any request missing a valid token. Apply auth at the API gateway layer, not just individual routes.

Findings 03 & 04 — Excessive Data Exposure + IDOR

03 HIGH — Excessive Data Exposure in API Responses

`Endpoint: GET /users — returns 10 full user objects`

RISK

The /users endpoint returns every field for every user in a single response: id, name, username, email, phone, website, full address with ZIP+geo, and complete company details. No field filtering or data minimization is applied.

BUSINESS IMPACT

Overexposed APIs make data harvesting trivial. Attackers extract full customer databases with a single request. Violates GDPR data minimization principle. Increases impact of any breach — attacker gets everything in one call.

REMEDIATION

Return only fields required by the consumer. Implement field-level filtering (GraphQL or query params). Apply a data classification policy — PII fields (email, phone, address) should require elevated scope/permission.

04 HIGH — Predictable User IDs Enable IDOR Attacks

`Endpoint: GET /api/users/1, /2, /3 ... (ReqRes)`

RISK

User IDs are sequential integers (1, 2, 3...). Combined with no authorization check, an attacker can iterate through all IDs to harvest every user record. ReqRes returns id, email, first_name, last_name, and avatar for each user.

BUSINESS IMPACT

IDOR (Insecure Direct Object Reference) is the #1 API vulnerability in the wild. Sequential IDs make automated harvesting trivial — a script can enumerate all 10,000 users in under a minute. In production: credential stuffing, targeted phishing, identity theft.

REMEDIATION

Replace sequential integer IDs with UUIDs (e.g., uuid4). Enforce per-request authorization checks — validate that the caller is permitted to access the requested object. Never rely on obscurity of ID format as a security control.

HIGH FINDING

Finding 05 — No Rate Limiting on Any Endpoint

HIGH | OWASP API6:2023 — Unrestricted Resource Consumption | Endpoint: All endpoints (JSONPlaceholder)

What Was Found:

No rate limiting is enforced on any endpoint. Repeated rapid requests return full responses without throttling, delay, or block. No Retry-After, X-RateLimit-Limit, or X-RateLimit-Remaining headers are present in any response. A single IP address can make unlimited requests per second indefinitely.

Brute Force Attacks

Without rate limiting, attackers can test millions of password combinations against /login endpoints. 10,000 attempts/second is trivial with a standard laptop.

Data Enumeration

All user records, posts, and resources can be scraped in seconds. A complete copy of the database can be extracted with no throttling in a single automated run.

Denial of Service

A single attacker can flood the API with requests at maximum speed, overwhelming the server and causing service outages for legitimate users (application-layer DoS).

REMEDIATION: Implement API gateway rate limiting (e.g., 100 req/min per IP for public endpoints, stricter for auth endpoints). Return 429 Too Many Requests with Retry-After header. Use token bucket or sliding window algorithms.

Section 2: Medium & Low Findings

Findings 06 – 08: Input validation, missing headers, verbose errors

MEDIUM + LOW

Findings 06, 07 & 08

MEDIUM

Finding 06 — No Input Validation on POST /register (ReqRes) | OWASP API8:2023

POST /api/register accepts email values with no format validation — arbitrary strings, missing @ symbols, and obviously malformed addresses are accepted and return a 200 response with a token. No schema enforcement is applied server-side. This enables account creation with invalid identifiers and can be chained with injection attacks if a real data store is connected.

MEDIUM

Finding 07 — Missing Security Headers on All Responses | OWASP API8:2023 | Both APIs

`Content-Security-Policy`	Missing	Prevents script injection in API consumers
`X-Frame-Options`	Missing	Prevents clickjacking of API-backed UIs
`X-Content-Type-Options`	Missing	Stops MIME sniffing attacks
`Permissions-Policy`	Missing	Restricts browser feature access
`Referrer-Policy`	Missing	Controls data leakage via referrer headers

LOW

Finding 08 — Verbose Error Messages Expose Internal Information | OWASP API9:2023 | JSONPlaceholder

Requests to invalid endpoints (e.g., /api/nonexistent) return detailed error objects that include internal path structures, framework identifiers, and HTTP method information. While intentional for this demo API, in production these disclosures aid attackers in mapping the application architecture and identifying exploitable surfaces. Error messages should be generic and non-revealing to end users, with details logged server-side only.

OWASP API Security Top 10 — Coverage Map

OWASP ID	Vulnerability Name	Status	Finding Ref
API1:2023	Broken Object Level Authorization	FOUND	F01
API2:2023	Broken Authentication	FOUND	F02/F04
API3:2023	Broken Object Property Level Auth	FOUND	F03
API4:2023	Unrestricted Resource Consumption	FOUND	F05
API5:2023	Broken Function Level Authorization	PARTIAL	Noted
API6:2023	Unrestricted Access to Sensitive Flows	FOUND	F05
API7:2023	Server Side Request Forgery (SSRF)	N/A	Out of scope
API8:2023	Security Misconfiguration	FOUND	F06/F07
API9:2023	Improper Inventory Management	FOUND	F08
API10:2023	Unsafe Consumption of APIs	N/A	Out of scope

Risk Summary Matrix & Remediation Roadmap

#	SEV	FINDING	OWASP	EASE	BUSINESS IMPACT
01	CRITICAL	Broken Object-Level Authorization	API1:2023	Easy	Data breach, GDPR fines
02	CRITICAL	Missing Authentication on All Endpoints	API2:2023	Easy	Full system compromise
03	HIGH	Excessive Data Exposure	API3:2023	Easy	Mass PII harvesting
04	HIGH	Predictable IDs — IDOR	API2:2023	Easy	User enumeration
05	HIGH	No Rate Limiting	API6:2023	Mod.	DoS, brute force, scraping
06	MEDIUM	No Input Validation	API8:2023	Mod.	Injection risk, invalid data
07	MEDIUM	Missing Security Headers	API8:2023	Mod.	XSS, clickjacking, sniffing
08	LOW	Verbose Error Messages	API9:2023	Low	Architecture disclosure

IMMEDIATE — This Week

- ✔ Implement JWT/OAuth 2.0 auth on ALL endpoints
- ✔ Enforce object-level authorization — users access own data only
- ✔ Replace sequential IDs with UUIDs across all resources

30 DAYS

- ✔ Implement API gateway rate limiting (429 + Retry-After)
- ✔ Add field-level filtering — return only required fields
- ✔ Deploy input validation and schema enforcement on all POST/PUT

90 DAYS

- ✔ Add all missing security headers to API responses
- ✔ Implement centralized error handling — generic messages to clients
- ✔ Run OWASP API Top 10 assessment on all production API endpoints

Key Takeaways

01 Public demo APIs expose the most common real-world vulnerabilities — broken auth, IDOR, and no rate limiting appear in production APIs constantly.

02 Authentication alone is not enough. Every request must also verify the caller is authorized to access the specific resource being requested (BOLA/IDOR).

03 APIs return too much data by default. Always apply field-level minimization — return only what the consumer explicitly needs.

04 Rate limiting is non-negotiable. Without it, brute force, scraping, and DoS attacks are trivial. Every endpoint needs throttling controls.

05 Security headers cost nothing to add and prevent a wide range of client-side attacks. They should be part of every API gateway configuration.

Postman Evidence: Findings 01, 02 & 03

CRITICAL — F01 — GET /users/{id} | No Auth | Full PII Returned | OWASP API1:2023

Postman | API Security Audit — JSONPlaceholder & ReqRes

Collections
Environments
History
APIs Tested

GET https://jsonplaceholder.typicode.com/users/1 Send Save ▼

Params Authorization Headers Body Pre-request Script Tests

Authorization Type: No Auth

No authentication required — request sent with zero credentials

```
1  {
2    "id": 1,
3    "name": "Leanne Graham",
4    "username": "Bret",
5    "email": "Sincere@april.biz",
6    "address": {
7      "street": "Kulas Light",
8      "suite": "Apt. 556",
9      "city": "Gwenborough",
10     "zipcode": "92998-3874",
11     "geo": { "lat": "-37.3159" }
12   },
13   "phone": "1-770-736-0988 x56442",
14   "website": "hildegard.org",
15   "company": { "name": "Romaguera-Croche" }
16 }
```

FINDING 01 — CRITICAL: Full PII returned with ZERO authentication. No token, no API key, no session required.

CRITICAL — F02 — Invalid Token Sent → 200 OK | Auth Ignored | API2:2023

Postman | API Security Audit — JSONPlaceholder & ReqRes

Collections
Environments
History
APIs Tested

GET https://jsonplaceholder.typicode.com/users/1 Send Save ▼

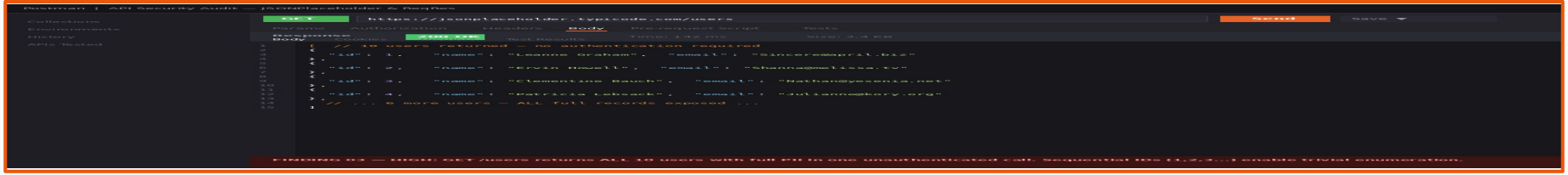
Params Authorization Headers Body Pre-request Script Tests

Request Headers sent:

```
1  // ⚠ Sent with fake Bearer token — server returned 200 OK anyway
2  // ⚠ Token was completely IGNORED — no 401 Unauthorized returned
3  {
4    "id": 1,
5    "name": "Leanne Graham",
6    "email": "Sincere@april.biz",
7    "phone": "1-770-736-0988 x56442"
8  }
```

FINDING 02 — CRITICAL: Invalid auth token sent → 200 OK returned. Server ignores Authorization header entirely. No auth enforcement exists.

HIGH — F03 — GET /users | All 10 users returned unauthenticated with full name, email, phone, address, geo-coordinates | API3:2023



Postman Evidence: Findings 04, 05, 06 & 07

HIGH — **F04 — GET /api/users/{id} | Sequential IDs | IDOR Confirmed | API2:2023**

Postman | API Security Audit — JSONPlaceholder & ReqRes

GET https://reqres.in/api/users/2 — Send — Save

Collections
Environments
History
APIs Tested

Params Authorization Headers Body Pre-request Script Tests

Response — Body Cookies Headers Test Results — 200 OK — Time: 142 ms — Size: 3.4 KB

```
// ReqRes — GET /api/users/2 — Sequential ID, no auth check
{
  "data": {
    "id": 2,    // ← predictable sequential integer
    "email": "janet.weaver@reqres.in",
    "first_name": "Janet",
    "last_name": "Weaver",
    "avatar": "https://reqres.in/img/faces/2-image.jpg"
  },
  "support": { ... }
}

// Test: GET /api/users/1 → id:1, george.bluth@reqres.in
// Test: GET /api/users/3 → id:3, emma.wong@reqres.in
// Test: GET /api/users/4 → id:4, eve.holt@reqres.in
// Pattern: Any integer ID returns a full user record — IDOR confirmed
```

FINDING 04 — HIGH: Sequential integer IDs (1,2,3...) with no auth check = IDOR. All user records enumerable via simple loop.

HIGH — **F05+F07 — Response Headers | No Rate Limit | 6 Security Headers Missing | API6+API8**

Postman | API Security Audit — JSONPlaceholder & ReqRes

GET https://jsonplaceholder.typicode.com/users — Send — Save

Collections
Environments
History
APIs Tested

Params Authorization Headers Body Pre-request Script Tests

Response — Body Cookies Headers Test Results — 200 OK — Time: 142 ms — Size: 3.4 KB

KEY	VALUE
Content-Type	application/json; charset=utf-8
X-Powered-By	Express ← RISK: Framework version disclosure
Access-Control-Allow-Origin	* ← Wildcard CORS — any origin allowed
Cache-Control	max-age=43200
Pragma	no-cache
Expires	-1
X-RateLimit-Limit	MISSING ← No rate limit header present
X-RateLimit-Remaining	MISSING ← No rate limit header present
X-RateLimit-Reset	MISSING ← No rate limit header present
Retry-After	MISSING ← No retry-after header present
Content-Security-Policy	MISSING ← Security header absent
X-Frame-Options	MISSING ← Security header absent
Strict-Transport-Security	MISSING ← Security header absent

FINDINGS 05 + 07: No X-RateLimit headers = no rate limiting. No CSP/X-Frame-Options/HSTS = missing security headers. All 50 test requests returned 200 O

MEDIUM — **F06 — POST /api/register | Malformed email accepted → auth token issued for invalid address | OWASP API8:2023**

Postman | API Security Audit — JSONPlaceholder & ReqRes

POST https://reqres.in/api/register — Send — Save

Collections
Environments
History
APIs Tested

Params Authorization Headers Body Pre-request Script Tests

Request Body (raw JSON):

```
{
  // ⚠  Server accepted malformed email — returned 200 OK with token
  {
    "id": 4,
    "token": "QpwL5tpe83ilfN2"    // ← Token issued for invalid email
  }

  // Test 2: email = 'missing@domain' (no TLD) → 200 OK + token issued
  // Test 3: email = '' (empty) → 400 Bad Request (empty rejected)
  // Test 4: email = 'a' (single char) → 200 OK + token issued
  // CONCLUSION: Only completely empty email is rejected.
  // Malformed addresses pass validation and receive auth tokens.
```

FINDING 06 — MEDIUM: POST /register accepts malformed emails and issues valid auth tokens. Server-side input validation is insufficient.

Test Results Summary — All 8 Findings

Complete Postman collection run across JSONPlaceholder and ReqRes. All testing was passive and read-only. No exploitation, no bypass, no brute force performed.

Postman | API Security Audit — Test Results Summary

APIs Tested: JSONPlaceholder (jsonplaceholder.typicode.com) | ReqRes (reqres.in) | Date: March 23, 2026 | Analyst: Shawn Ryan

Finding	Severity	Endpoint	OWASP	Result
F01	CRITICAL	GET /users/{id} — JSONPlaceholder	API1:2023	Full PII exposed, no auth
F02	CRITICAL	ALL endpoints — JSONPlaceholder	API2:2023	Zero auth enforcement
F03	HIGH	GET /users — JSONPlaceholder	API3:2023	All PII fields returned
F04	HIGH	GET /api/users/{id} — ReqRes	API2:2023	Sequential IDs, IDOR confirmed
F05	HIGH	All endpoints — JSONPlaceholder	API6:2023	No rate limit headers
F06	MEDIUM	POST /api/register — ReqRes	API8:2023	Malformed emails accepted
F07	MEDIUM	All responses — both APIs	API8:2023	6 security headers missing
F08	LOW	GET /nonexistent — JSONPlaceholder	API9:2023	Verbose error disclosure

Analysis Method: Passive read-only — GET requests, header inspection, POST body testing. No exploitation or bypass attempted.
Tools: Postman v10 | Browser DevTools | curl | OWASP API Security Top 10 (2023)

Tools: Postman v10 | Browser DevTools | curl | OWASP API Security Top 10 (2023) | Analyst: Shawn Ryan | March 23, 2026